

ISP Network Planner — Project Report

Course: CS3101 Design & Analysis of Algorithms **Project:** Planning a mobile/internet network for a real city

Repository: https://github.com/rajshinde0/ISP_NP

Abstract

This project plans an ISP's network end to end: given customer locations and candidate tower sites on a real road network, it decides **which towers to build, what equipment to put on them, how to assign customers, how to wire the towers together, where the bandwidth bottleneck lies, and how signal routes to a customer**. Each stage is deliberately built on a different algorithm from the syllabus, and every algorithm is hand-written — NetworkX is used only as a graph container and as an independent oracle in the test suite.

The centrepiece is **Stage 1**, which is NP-hard. We implement both an exact branch-and-bound solver and a greedy approximation, and measure the trade-off directly: unpruned backtracking grows by a factor of roughly 16 for every four extra candidate sites and hits a 20-second wall at

$|C| = 28$, while branch and bound stays under 6 ms; greedy runs in a tenth of a millisecond but

costs up to 62% more than the optimum.

Where a simple rule and a principled algorithm both existed, we implemented both and measured which wins. In three cases the measurement **contradicted the intuition** that motivated the work, and Section 8 reports those honestly rather than quietly keeping the flattering result.

1. Problem statement

An ISP entering a new service area must answer six coupled questions. We model them as follows.

Question	Formal problem
Which towers to build?	Minimum-cost set cover over demand points
What equipment on each?	0/1 knapsack under a per-tower budget
Which customers to which tower?	Maximum flow on a bipartite capacity network
How to wire the towers?	Minimum spanning tree over a road-distance metric closure
Where does bandwidth run out?	Maximum flow / minimum cut
How does signal reach a customer?	Single-pair shortest path

The coupling matters and is a recurring theme: the tower set determines the customer assignment, which determines equipment sizing, which determines link capacities, which determines the bottleneck. An error early does not stay local — Section 8.2 documents a case where one greedy choice corrupted two downstream stages.

Scope. Inputs are a road network (real, via OpenStreetMap, or a synthetic grid for offline work), a set of weighted demand points, and a set of candidate tower sites with build costs and coverage radii. Output is a complete build plan with costs, capacities, cable length, the bottleneck, and per-customer routing.

2. System overview

Stage 0	road graph + demand + candidate sites	BFS connectivity
Stage 1	which towers to build	EXACT: branch & bound APPROX: greedy
Stage 2	equipment per tower	0/1 knapsack DP
Stage 2b	customers -> towers	GREEDY: nearest OPTIMAL: max-flow
Stage 3	backbone + resilience + exchange	Prim Kruskal, Tarjan, 1-median
Stage 4	bandwidth from exchange to towers	Edmonds-Karp max-flow / min-cut
Stage 5	route signal to a customer	Dijkstra A*
Stage 6	dashboard	Streamlit + Folium

Five stages offer **two algorithms side by side** — exact vs approximate, greedy vs optimal, uninformed vs heuristic. This is deliberate: the project's purpose is to demonstrate algorithmic trade-offs, so each comparison is computed live and reported rather than asserted.

2.1 Code structure

File	Lines	Contents
planner.py	877	All algorithms, stages 0–5
app.py	257	Stage 6: Streamlit + Folium dashboard
interface.py	67	The locked data contract between team members, plus all tunable constants
bench.py	158	Stage-1 benchmark CLI; emits the report's figures
tests/	1,294	99 tests against NetworkX and brute-force oracles
docs/np-hardness.md	258	The NP-hardness treatment summarised in Section 5

3. Algorithm design and analysis

Notation: `V`, `E` for road-network nodes and edges; `D` for demand points; `C` for candidate sites; `k` for built towers; `B` for a budget in k\$.

3.1 Stage 0 — Road network and connectivity (Unit I)

The road graph is either fetched from OpenStreetMap and projected to metres, or generated as a 25×25 synthetic grid (625 nodes, 1,110 edges) for offline work. Both carry `x / y` in **metres** for distance arithmetic and `lat / lon` in degrees for display.

`components()` is a hand-written **BFS** in $O(V+E)$, used to discard disconnected road fragments — an island of road with no route to the rest would silently break every later stage.

Coordinate correctness. Coverage radii are compared in projected metres, never in degrees. This was flagged as a project risk at planning time and is the reason the OSM path projects before computing anything. A related subtlety:

OSMnx stores x as *longitude* and y as *latitude*, so flattening its graph requires a deliberate swap. Getting it backwards is silent — it relocates every tower — so it is covered by a dedicated test.

3.2 Stage 1 — Which towers to build (Units IV and VI)

The headline problem, and the only NP-hard one. Formally **minimum-cost set cover**: choose $X \subseteq C$ minimising $\sum \text{cost}(i)$ subject to $\bigcup S_i = D$, where S_i is the set of demand points within tower i 's radius. Coverage sets are held as **bitmasks**, so union and intersection are single machine words up to 64 demand points.

Greedy approximation. Repeatedly take the candidate maximising *newly covered weight per unit cost*. $O(|C|^2 \cdot |D|/64)$. Guarantees a solution within $H(d) \leq \ln n + 1$ of optimal (Section 5.3).

Exact branch and bound. Depth-first search over include/exclude decisions, with three pruning rules, each of which only ever discards branches that cannot contain a strictly better solution:

1. **Greedy warm start** — seed the incumbent with the greedy answer, so the bound is useful at the root instead of infinite.
2. **Feasibility cuts** — abandon a branch when its partial cost already meets the incumbent, or when the union of all remaining candidates cannot complete the cover.
3. **Fractional lower bound** — take the best cost-per-weight ratio ρ still available; covering the remaining weight w cannot cost less than ρw . This is a relaxation (it permits taking sets fractionally), so it never over-estimates.

Worst case remains $O(2^{|C|})$. A configurable time limit returns the best incumbent found and **flags it as non-optimal** — Section 6.1 shows why that flag matters.

3.3 Stage 2 — Equipment per tower (Unit III)

A 0/1 knapsack DP per built tower: $\text{dp}[i][b]$ = maximum capacity using the first i equipment options within budget b . $O(n \cdot B)$ per tower.

Two refinements beyond the textbook form:

- The allowance is **tier-scaled** — a high-tier tower covers 2.25× the radius, so it gets a larger budget.
- The purchase is **trimmed to demand**: rather than always emptying the budget, the DP table is scanned for the cheapest budget level meeting $\text{HEADROOM} \times$ the tower's own load, so a lightly loaded tower is not over-equipped. A tower with no customers of its own still buys one radio, because it may be needed purely as a backbone relay.

A precision for the analysis. Knapsack is NP-hard, but this DP is *pseudo*-polynomial: $O(n \cdot B)$ is exponential in the input size, since B is encoded in $\log B$ bits. It is fast here only because B is a small integer. Set cover, by contrast, is **strongly** NP-hard and admits no such DP unless $P = NP$.

3.4 Stage 2b — Assigning customers (Unit V)

The obvious rule — send every customer to their nearest covering tower — ignores capacity. We implement it (it is needed as a bootstrap, below) and an optimal alternative.

Assignment is modelled as **maximum flow** on a bipartite network:

```
SRC —  $w_j \times \text{mbps}$  —> customer  $j$  —> every tower covering  $j$  —  $\text{caps}[k]$  —> SINK
```

All capacities are integral Mbps. The maximum flow is the most bandwidth the built network can actually deliver; each tower's share is read off its sink arc. Reuses the same Edmonds–Karp implementation as Stage 4.

Breaking the circular dependency. Equipment is sized from the load, but a capacity-aware assignment needs the capacities. Two phases resolve it: the greedy pass provides a provisional load to purchase against, then max-flow reassigns optimally against what was bought.

Stated limitation. Max-flow maximises *total* customers served and is indifferent between tied optima, so a customer may be routed to a farther tower than necessary. Correcting that requires a min-cost flow, which we judged out of scope — Unit V is already demonstrated, and the objective the dashboard reports is customers served.

3.5 Stage 3 — Backbone, resilience, exchange (Units I, II, IV)

Metric closure. Towers are wired along *roads*, not straight lines, so we first build a $k \times k$ matrix of shortest road distances — k Dijkstra runs. *A is deliberately not** used here: it needs a single target, and this needs all of them.

Minimum spanning tree, two ways.

	Strategy	Complexity	Better when
Prim	grow one tree from a seed	$O(k^2)$ with array scan	dense graphs — like this closure
Kruskal	sort all edges, reject cycles	$O(k^2 \log k)$, sort-dominated	sparse graphs

Kruskal uses a hand-written **disjoint-set forest** with path compression and union by rank — effectively constant per operation (inverse Ackermann). Both produce the same total length on every instance tested; which runs is a demonstration, not a decision.

Resilience (Tarjan bridge-finding, $O(V+E)$). A spanning tree is *entirely* bridges — cut any link and the backbone splits — so redundant links are the only thing buying resilience. We find what remains using DFS low-link values, implemented iteratively so a long chain of towers cannot exhaust the Python stack. Parallel links are tracked by **edge id** rather than parent node: two towers joined by two fibres genuinely have no single point of failure between them, and the textbook skip-the-parent shortcut wrongly reports one.

Siting the exchange. The exchange is the head-end where upstream fibre lands. Three rules, all exact by enumeration because the exchange must sit at one of the k towers — unlike general k -median, which is NP-hard:

- **1-median** (default) — least total road distance, weighted by each tower's customer load.
- **1-center** — least worst-case distance.
- **Centroid** — nearest the Euclidean centre of the towers; the original rule, kept as baseline.

3.6 Stage 4 — Bandwidth and the bottleneck (Unit V)

Maximum flow from the exchange to the customers, by **Edmonds–Karp** — Ford–Fulkerson with the augmenting path chosen by BFS, which is what bounds the iteration count at $O(V \cdot E^2)$. Plain Ford–Fulkerson has no such bound; the report should not claim it.

Node splitting. Each tower k becomes $(in, k) \rightarrow (out, k)$ carrying its equipment capacity, so a tower cannot relay more traffic than its own hardware switches. Without the split, a degree-4 exchange could emit four times its own capacity. Backbone links attach $out \rightarrow in$; each tower drains its customers from out . The upstream fibre is fed to $(out, exchange)$, because it lands on the backbone switch rather than passing through that tower's access radio.

By the **max-flow min-cut theorem**, the minimum cut names the bottleneck exactly. The dashboard translates cut arcs back into plan language: a saturated backbone link, or a tower limited by its own equipment.

3.7 Stage 5 — Routing to a customer (Unit II)

Dijkstra with a binary heap, $O((V+E) \log V)$, and **A*** with a straight-line heuristic in projected metres.

The heuristic is **admissible** — a road between two points is never shorter than the line between them — so *A returns Dijkstra's exact optimum, not an approximation. It is also consistent**, so no node is ever re-expanded; the implementation asserts this invariant directly.

On the synthetic grid the heuristic is not merely admissible but *exact*, since each edge length is the Euclidean distance between its endpoints. That is why the saving there is large, and why Section 6.4 reports the saving across several query patterns rather than quoting one number.

4. Complexity summary

Stage	Algorithm	Time	Space	Class
0	BFS connectivity	$O(V+E)$	$O(V)$	P
1	Greedy set cover	$O(C ^2 \cdot D /64)$	$O(C)$	P, $(\ln n + 1)$ -approx
1	Branch & bound set cover	$O(2^{ C })$ worst case	$O(C)$	NP-hard
2	Knapsack DP	$O(n \cdot B)$ per tower	$O(n \cdot B)$	<i>pseudo</i> -polynomial
2b	Max-flow assignment	$O(V \cdot E^2)$	$O(V+E)$	P
3	Road-distance matrix	$O(k(V+E) \log V)$	$O(k^2)$	P
3	Prim MST	$O(k^2)$	$O(k)$	P
3	Kruskal MST + union-find	$O(k^2 \log k)$	$O(k)$	P
3	Tarjan bridges	$O(V+E)$	$O(V)$	P
3	Exchange siting	$O(k^2)$	$O(1)$	P
4	Edmonds–Karp	$O(V \cdot E^2)$	$O(V+E)$	P
5	Dijkstra	$O((V+E) \log V)$	$O(V)$	P
5	A*	$O((V+E) \log V)$	$O(V)$	P

Stage 1 is the only entry that is not polynomial — which is precisely why the project makes it the centrepiece.

5. NP-hardness of Stage 1 (Unit VI)

Full treatment in `docs/np-hardness.md` ; summarised here.

5.1 The decision problem is NP-complete

DECIDE(D, S, c, k): is there a cover of total cost $\leq k$?

In NP: a candidate cover is a certificate verifiable in $O(|C| \cdot |D|)$ — OR the masks, compare against the full mask, sum the costs.

NP-hard, by reduction from SET-COVER (Karp, 1972). Given a SET-COVER instance (U, F, k) , set $D := U$, $S := F$, every cost to 1, budget k . Then total cost $\leq k$ exactly when at most k sets are used, so the instances are yes-instances together.

The reduction is an identity map plus unit costs — clearly polynomial. Hence SET-COVER $\leq_p$ DECIDE.

Note the direction: the *known-hard* problem reduces **to** ours, establishing that ours is at least as hard. The reverse would prove nothing.

5.2 The hardness survives our restriction

This is the step most easily missed. Our generator never produces arbitrary set systems — every coverage set is a **disc**. Restricting an instance family can make a hard problem easy: set cover is NP-hard in general but polynomial when the sets are intervals on a line.

It survives here. Covering points in the plane by discs is itself NP-hard (Fowler, Paterson & Tanimoto, 1981), and our version is more general still — two radii rather than one, and non-uniform costs. The hardness is a property of the real problem, not an artifact of the implementation accepting arbitrary bitmasks.

Caveat worth knowing: geometric covering is easier to **approximate** than general set cover — unit-disc cover admits a PTAS. We do not use one because our discs are not unit (two tiers, a 2.25× radius ratio) with non-uniform costs, and because exact branch and bound finishes in milliseconds at the sizes demonstrated.

5.3 What greedy guarantees, and why it is the right approximation

Chvátal (1979): cost-weighted greedy returns at most $H(d) \cdot \text{OPT}$, where $H(n) = \sum_{k=1}^n 1/k \leq \ln n + 1$ and d is the largest set size.

Proof sketch. When greedy takes a set of cost c newly covering t elements, charge c/t to each. For any set S^* in an optimal cover with elements $e_1 \dots e_p$ in the order greedy covered them: when e_j was covered, S^* still had $\geq p-j+1$ uncovered elements, so it offered a ratio of at most $c(S^*)/(p-j+1)$ — and greedy chose at least as well. Summing gives $\sum \text{charge}(e_j) \leq c(S^*) \cdot H(p)$, and summing over the optimal cover gives the bound. ■

Our greedy ranks on *weight* per cost, which is the same algorithm on an instance where a demand point of weight w is w copies — so d is the largest set **weight**. (An earlier version of our test applied the unit-weight form $H(|D|)$, which is not guaranteed for weighted instances; it passed on every seed tried but was unsound, and was corrected.)

And no polynomial algorithm does better. Feige (1998) showed no polynomial $(1-\epsilon) \ln n$ approximation exists unless $\text{NP} \subseteq \text{DTIME}(n^{O(\log \log n)})$; Dinur & Steurer (2014) strengthened the assumption to $P \neq \text{NP}$. Greedy's $\ln n + 1$ is therefore essentially the ceiling for any polynomial-time algorithm — which is the real argument for shipping it alongside the exact solver, rather than treating it as a convenient shortcut.

6. Results

All measurements on the synthetic 25×25 grid (625 nodes, 1,110 edges), 60 demand points, seed 0, 5 Mbps per customer, 12 k\$ base equipment budget, 2 redundant links.

6.1 Stage 1 — the headline trade-off

Naive backtracking is the **identical recursion with all three pruning rules removed**, so node counts compare directly. 20-second limit.

C	naive nodes	naive time	B&B nodes	B&B time	ratio	exact k\$	greedy k\$	gap
8	511	0.0002 s	22	0.0001 s	23×	67.26	67.26	0.0%
12	8,191	0.004 s	38	0.0002 s	216×	76.61	84.72	10.6%
16	127,935	0.052 s	85	0.0005 s	1,505×	77.40	91.16	17.8%
20	1,946,815	0.850 s	456	0.0030 s	4,269×	80.93	110.24	36.2%
24	30,799,999	13.13 s	676	0.0040 s	45,562×	77.31	85.41	10.5%
28	46,284,591	timed out	572	0.0053 s	—	74.14	96.24	29.8%

Three findings:

1. **The 2ⁿ curve is directly visible.** Naive node counts multiply by roughly 16 for every four extra candidates — that is 2⁴ — and the search hits the wall at |C| = 28.
2. **Branch and bound does not blow up** at these sizes: under 6 ms through |C| = 28, and 0.12 s at |C| = 40. This does **not** contradict NP-hardness — the worst case is untouched. The bounds simply happen to be strong on geometric instances, where most candidates are either clearly worth taking or clearly dominated.
3. **A timed-out search is not an optimum, and this table proves it.** At |C| = 28 naive's incumbent when the clock expired was 86.32 k\$, while branch and bound *proved* the optimum to be 74.14 k\$. Timed-out points are marked accordingly and never reported as optimal.

Across seeds 0–9 at |C| = 20, **greedy is strictly costlier than the optimum on 8 of 10 instances, by up to 62%.**

6.2 Stage 2b — capacity-aware assignment

	nearest tower	max-flow
T0 load against its 1,300 Mbps of kit	1,900 Mbps — 146%	1,300 Mbps — 100%
T1–T5 utilisation	46–50%	46–50%
Total claimed served	3,315 Mbps	2,715 Mbps
Shortfall reported	<i>none</i>	600 Mbps

The greedy rule was not merely unbalanced — it was **wrong about what the network delivers**, claiming 3,315 Mbps when the purchased equipment can carry 2,715.

The diagnosis is sharper than "rebalance it". T0 has 1,900 Mbps of demand **no other built tower can reach**, and 1,300 Mbps is the most any budget can purchase, so 600 Mbps is unservable by any assignment; only 540 Mbps of demand lies in range of more than one tower. The planning conclusion is to build another tower in that area — not to buy a bigger radio.

6.3 Stage 3 — resilience and exchange siting

At the default settings the backbone has 7 links over 14.01 km and **one remaining single point of failure**, link T0–T4. Nothing reported this before Tarjan bridge-finding was added.

Exchange siting proved to matter far more than distance alone suggests, because the exchange is where *all* traffic enters. Max-flow delivered, by rule, over 6 seeds:

seed	1-median	1-center	centroid
0	92%	92%	63%
1	60%	54%	54%
2	100%	92%	100%
3	96%	96%	96%
4	100%	100%	82%
5	100%	100%	100%
best in	6 / 6	4 / 6	3 / 6

Seed 0 is the instructive case: the centroid rule picks a **shorter** total distance (12.24 km vs 13.92 km) and delivers **far less** bandwidth (63% vs 92%). Minimising cable to the exchange is the wrong objective — what matters is sitting near the *demand*, since that determines how much traffic must cross the backbone at all. Hence the load weighting.

Prim, Kruskal and NetworkX agree on the MST weight to the metre on every instance tested (8,825.6 m at the default).

6.4 Stage 5 — what the A* heuristic buys

Identical distances throughout; the figures are nodes expanded.

Query pattern	Dijkstra	A*	saved
A built tower to a demand point (what the app runs)	2,012	302	85%
40 uniformly random node pairs	14,751	3,997	73%
Every tower to the first six demand points	12,357	3,933	68%
Opposite corners of the grid	2,186	1,949	11%

The last row is the interesting one. *A wins by being directed**, so when the target lies on the far side of the map almost every node is on a plausible route and there is little left to prune. This is also why the k Dijkstra runs in Stage 3 were left alone — they have no single target for a heuristic to aim at.

7. Verification

"It runs" is not evidence when the entire premise is that the algorithms are hand-written. The project carries **99 tests**, each algorithm checked against an independent oracle.

Algorithm	Oracle
<code>components</code> , <code>largest_component</code>	<code>nx.connected_components</code> — the partition, not just the count
<code>dijkstra</code>	<code>nx.dijkstra_path_length</code> ; the path is re-walked and its lengths re-summed
<code>astar</code>	<code>dijkstra</code> and NetworkX, plus the no-re-expansion invariant
<code>prim</code> , <code>kruskal</code>	each other and <code>nx.minimum_spanning_tree</code> on the same closure
<code>UnionFind</code>	a naive set-of-sets reference over 400 random union sequences
<code>bridges</code>	<code>nx.bridges</code> over 300 random graphs, plus path / cycle / barbell shapes
<code>edmonds_karp</code>	<code>nx.maximum_flow_value</code> over 300 random networks, CLRS fig. 26.1, and max-flow = min-cut
<code>assign_customers_flow</code>	<code>nx.maximum_flow_value</code> on the same bipartite network
<code>knapsack</code> , <code>exact_cover</code>	exhaustive subset search
<code>greedy_cover</code>	its own $H(d)$ guarantee, plus an instance separating cost-weighted from unweighted
<code>choose_exchange</code>	brute force over all k candidates

MST weight, not edge set. The minimum spanning tree's *weight* is unique even where the tree is not, so the cross-validation asserts on weight. Asserting equal edge sets would be wrong in general, even though they coincide on our instances.

7.1 Mutation testing

Tests can pass for the wrong reason, so the suite was itself tested: **46 deliberate bugs** were injected into `planner.py` across the project — 10 during the initial review and 36 while adding the five comparison algorithms — and the suite re-run against each. A surviving mutation was treated as a question, not a verdict — each was investigated rather than assumed to be a gap:

- **Real gaps (fixed).** A hand-built distance matrix was degenerate — `sum` and `max` selected the same tower — so a 1-median that secretly computed `max` passed; one test was entirely vacuous. Rebuilt on a matrix where the rules genuinely disagree. Separately, a 6-node network was found by random search where omitting the residual reverse edge in Edmonds–Karp silently returns flow 7 instead of 9; it is now pinned as a regression test.
- **Provable no-ops (documented, not "fixed").** Verified over 40,000 random multigraphs that using `low[v]` instead of `disc[v]` on a back edge changes no bridge result, and that self-loops need no special case. Verified over 5,000 random sequences that inverting union-by-rank changes no correctness outcome, and that Prim's `key[0] = 0` is documentary — every key starts at infinity, so the first `min()` selects index 0 regardless. Each is commented in place so it is not later "corrected" into a genuine bug.

After both rounds, every mutation representing a real defect is caught.

8. What the measurements overturned

Three results contradicted the reasoning that motivated the work. They are reported because the contradiction is itself a finding.

8.1 The exact solver was *too* effective to demonstrate what we claimed

Our plan predicted backtracking would "stall past ~15–20 candidate sites". It does not: branch and bound solves $|C| = 40$ in 0.12 s. The pruning is simply too strong, so the intended 2ⁿ demonstration never materialised — the runtime chart was two flat lines.

The fix was the opposite of slowing the solver down: add **unpruned** backtracking as a third curve. That is the honest 2ⁿ demonstration, and it is a better artifact — it shows not that the problem is hard in principle, but exactly what pruning buys, up to a 45,562× reduction.

8.2 One greedy choice corrupted two downstream stages

The nearest-tower assignment was chosen for simplicity. Because its output feeds *both* equipment sizing and the Stage 4 demand figures, its error did not stay local: it over-subscribed a tower to 146% and reported full service while 600 Mbps was undeliverable. Coupled pipelines propagate bad local decisions, and only replacing the rule with an optimal one exposed it.

8.3 A predicted improvement did not appear; an unpredicted one did

We expected bridge-targeted redundancy to be clearly better. Sweeping 225 configurations, the two placement rules differ in only **29 (13%)** — the cheapest link usually happens to cover a bridge anyway, and on the default instance they are identical. The tests therefore assert the invariant that bridge-targeting never leaves *more* failure points, rather than an improvement that does not always exist. The *reporting* of single points of failure proved the larger win.

Conversely, exchange siting was justified on distance and turned out to matter for **bandwidth** — moving the exchange alone took the console pipeline from 65% to 100% of demand served.

9. Limitations and future work

- **The OpenStreetMap fetch is not verified end to end.** `graph_from_place` requires a boundary *polygon*, and Nominatim returns bare *points* for most neighbourhood names — all four Pune neighbourhoods tested failed this way. A `graph_from_address` fallback with a radius was added, and everything around the fetch is tested offline, but Overpass was too slow from our development machine to confirm a live download.
- **Assignment ignores distance among tied optima.** Max-flow maximises total served; preferring nearer towers among equal-value solutions needs a min-cost flow.
- **Equipment sizing is a two-phase heuristic**, not a joint optimum. Sizing and assignment are circular; we break the cycle rather than solve it jointly.
- **Coverage is a disc.** Real radio propagation is shaped by terrain and buildings.
- **A single exchange.** Multiple head-ends would make siting a genuine k-median problem — NP-hard, and a natural extension of the Stage 1 material.

10. Contributions

Per the module split agreed at planning time:

Area	Owner
Stage 0 — road graph, demand generation, connectivity	Person A
Stage 1 — greedy and branch-and-bound facility location	Person A
Stage 2 — knapsack equipment selection	Person A
Stage 3 — MST backbone	Person B
Stage 4 — max-flow and min-cut	Person B
Stage 5 — shortest-path routing	Person B
Stage 6 — dashboard integration	Person B
Theory, benchmarks, report	Both

The single rule that prevented integration failure was agreeing the shared data format **before** either member wrote stage code. That contract lives in `interface.py`, including a table of where the implementation's field names deviate from the original specification — so the divergence is recorded rather than discovered during integration.

References

1. R. M. Karp (1972). *Reducibility Among Combinatorial Problems*. Complexity of Computer Computations. — set cover NP-completeness.
2. V. Chvátal (1979). *A Greedy Heuristic for the Set-Covering Problem*. Mathematics of Operations Research 4(3). — the $H(n)$ bound for weighted set cover.
3. R. J. Fowler, M. S. Paterson, S. L. Tanimoto (1981). *Optimal packing and covering in the plane are NP-complete*. Information Processing Letters 12(3).
4. U. Feige (1998). *A Threshold of $\ln n$ for Approximating Set Cover*. Journal of the ACM 45(4).
5. I. Dinur, D. Steurer (2014). *Analytical Approach to Parallel Repetition*. STOC.
6. J. Edmonds, R. M. Karp (1972). *Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems*. Journal of the ACM 19(2).
7. R. E. Tarjan (1974). *A Note on Finding the Bridges of a Graph*. Information Processing Letters 2(6).
8. P. E. Hart, N. J. Nilsson, B. Raphael (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics 4(2). — A*.
9. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*.
10. G. Boeing (2017). *OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks*. Computers, Environment and Urban Systems 65.

Citation details should be checked against your course's referencing requirements before submission.

Appendix — Running the project

```
python -m venv venv
venv\Scripts\activate      # Windows
pip install -r requirements.txt

python planner.py           # console pipeline, all stages
streamlit run app.py        # full dashboard
pytest -q                  # 99 tests, ~6 s
python bench.py             # regenerate the Stage 1 figures
```

Select "Synthetic grid (offline)" in the sidebar to run without a network connection.